

Panadería La Espiga Dorada — control de operación

Caso práctico Consultor Developer · Ocean X

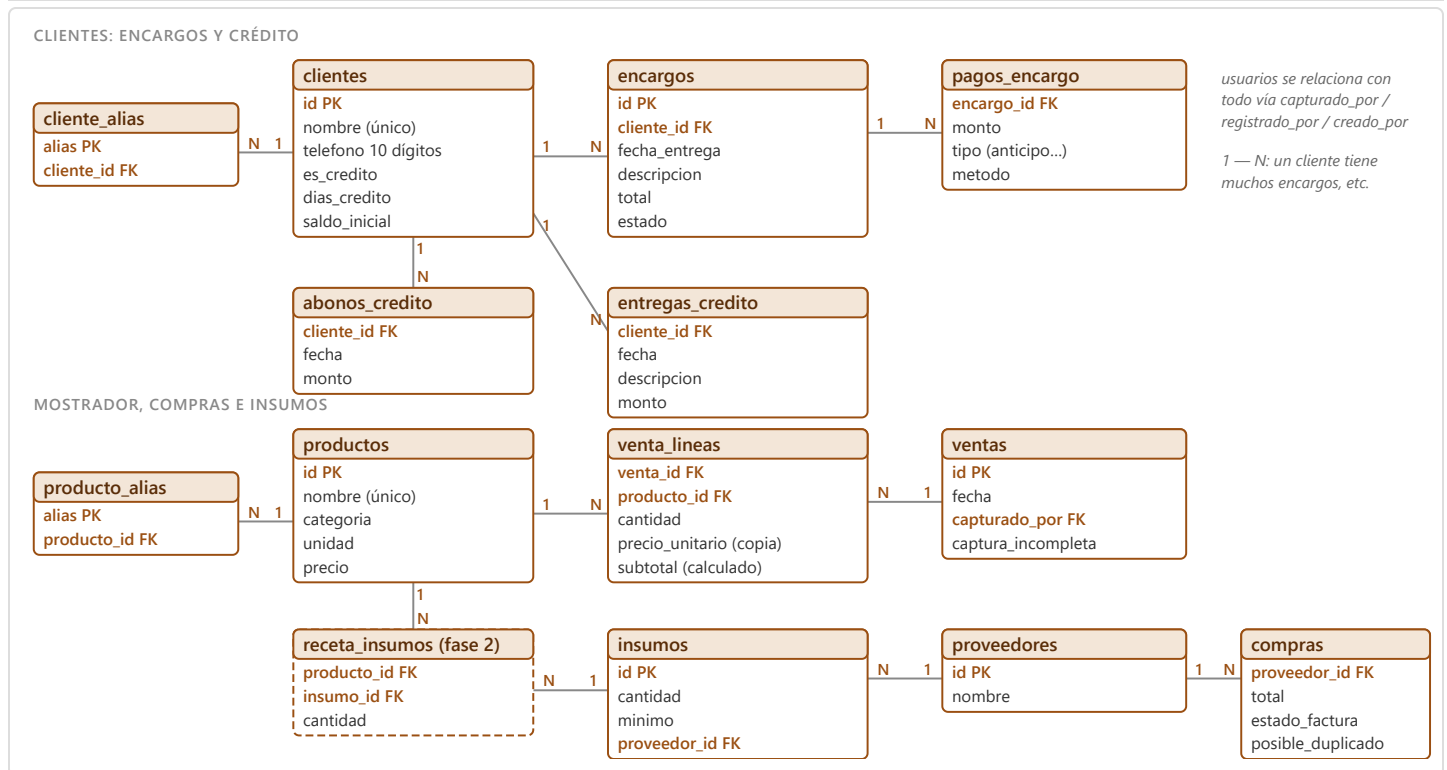
Adrián Galván Salgado · 9 de octubre de 2026

App: <https://green-coleslaw-perkiness.ngrok-free.dev> · Repo:

github.com/A01285442/la-espiga

Usuarios de prueba: enviados en el mensaje de entrega (no en este documento)

1. Cómo modelé los datos



- **Un cliente es un cliente** aunque esté escrito de 4 formas: una fila en **clientes** y sus variantes en **cliente_alias** (17 formas → 6 negocios). Igual con productos (20 nombres → 15). Un mismo cliente puede encargar pasteles y tener crédito (la Escuela, El Portal).
- **Los saldos nunca se guardan: se calculan** desde los movimientos. Encargo: total – pagos. Crédito: saldo inicial + entregas – abonos. No puede existir un "falta 200" que no cuadre.
- **La base hace cumplir las reglas:** CHECK (montos > 0, teléfono de 10 dígitos, entrega ≥ pedido) y triggers (no cobrar más que el total, precio tomado del catálogo, mostrador no cancela encargos). Cálculos en vistas (**v_encargos**, **v_saldos_credito**, **v_ventas_diarias**...). 15 tablas; **usuarios** se relaciona con todo vía **capturado_por** / **registrado_por**.
- **Limpieza reproducible** (**seed/limpiar_y_cargar.py**): fechas texto, "\$16", "520 pesos", totales vacíos; si aparece un nombre desconocido el script se detiene en vez de adivinar. Cada decisión queda en **docs/reporte_limpieza.md**.

2. Qué construí, en qué orden y por qué

0. **Esquema, permisos y carga del Excel.** Modelo y seguridad son lo que más pesa, y todo depende de datos limpios.
1. **Encargos con "Urgentes".** Es donde ya se perdió dinero (2 pasteles) y donde la información vive en el teléfono de Lupita. Anticipo y saldo, cobro al entregar, atrasados en rojo, lista para Toño por WhatsApp.
2. **Clientes de crédito.** \$15,660 vencido al 9 de octubre. Tabla de clientes con botón Recordar (WhatsApp con monto ajustable), estado de cuenta, días de crédito por cliente (La Fogata 15).
3. **Caja + Resumen.** Ticket por cliente con buscador y secciones del catálogo, total en pantalla y un clic para cobrar (cada cobro es una venta); y el resumen que Carmen hoy arma sumando columnas.
4. **Compras e insumos** (quedó tiempo): duplicados por confirmar, facturas por pedir, conteo de bodega con mínimos y aviso de qué pedir.

Hallazgo: el Excel de septiembre registra **\$92,860** (mostrador \$36,790 + encargos \$29,460 + crédito \$26,610) contra ~\$180,000 que Carmen factura, y compras por \$108,339. La mitad de la venta nunca se anota: por eso "no le cuadra con la caja". Hay 3 días marcados "se fue la luz".

3. Qué dejé fuera a propósito y cómo seguiría

Fuera	Por qué / segunda etapa
Costo por pastel y margen (petición 5)	Sin recetas no hay costo honesto; inventarlo sería peor que no tenerlo. Ya existe la tabla <code>receta_insumos</code> . Etapa 2: 1 h con Toño y Memo para los 5 productos que más venden, capturar en compras el precio por insumo (kg de harina, huevo) y calcular costo por pieza y margen.
Avisos automáticos (WhatsApp/correo)	Hoy los avisos están en pantalla y con botones <code>wa.me</code> (sin costo). Etapa 2: un mensaje diario programado a Carmen (cobros, insumos) y a Toño (encargos de mañana).
Corte de caja	Registrar efectivo al cierre y compararlo con lo capturado: es lo que hará que "le cuadre la caja".
Operación sin luz / sin internet	Captura en papel con folio y recaptura; o app instalable con modo offline.
Hosting	Hoy corre en Docker en mi computadora y sale a internet por un túnel HTTPS (ngrok), para no generar costos. Etapa 2: el mismo <code>docker-compose</code> en un servidor, o Supabase + Vercel (el esquema ya es Postgres).
Detalles	Rol "producción" para Toño; que recargar la página no cierre sesión (cookie firmada); edición de catálogo de productos y precios desde la app.

4. Supuestos donde Carmen fue ambigua y qué le preguntaría

Supuestos (todos están en el código y en el reporte de limpieza)

- Los negocios abonan a su cuenta, no a una nota: el pago cubre primero lo más viejo.
- Crédito a 7 días (cobro semanal), La Fogata a 15. Don Chuy inicia con \$2,400 de agosto, marcado "por confirmar".
- "SI", "pagó en efectivo", "transferencia" = se liquidó al entregar. "falta 200" aparece 5 veces y nunca cuadra con los números: respeté los números y lo marqué para revisar.
- Encargos con fecha pasada = entregados. Mostrador sí ve el saldo de un encargo (lo cobra al entregar), no el crédito ni los totales.
- "Conchas" sin sabor se guardan aparte y ya no se pueden capturar. Compras sospechosas se marcan, no se borran.

Las 3 preguntas por WhatsApp

No las usé: preferí avanzar con supuestos explícitos y fáciles de cambiar. Si las hubiera usado, serían las que cambian el modelo o los permisos:

- ¿Los negocios pagan notas específicas o abonan a lo que deben? (*cuenta corriente vs. nota por nota*)
- ¿Lupita y Karla cobran el saldo de un pastel al entregarlo? (*si mostrador ve el saldo*)
- ¿Cuánto debe Don Chuy de agosto? (*saldo inicial real*)

Para la siguiente visita

- ¿Los 2 pares de clientes con el mismo teléfono son la misma familia?
- ¿Dónde está la libreta de encargos de agosto?
- ¿Las 2 cargas de gas del 16/09 y los 3 tickets de Costco del 19/09 son reales?
- ¿Quién captura el conteo de bodega? ¿Qué mínimo quiere para la manteca?

5. Cómo protegí los datos

- La seguridad está en la base, no en los botones.** Tres roles de Postgres: `app_login` (solo puede ejecutar `autenticar()`), `app_mostrador` y `app_duena`. La app se conecta con el rol de quien inició sesión. Permisos `GRANT` por tabla y por columna: mostrador recibe *permission denied* si intenta leer compras, crédito, insumos o totales de venta, borrar, o cambiar el total de un encargo.
- Vistas con `security_invoker` (si no, una vista se salta los permisos). Contraseñas con `bcrypt (pgcrypto)`; nadie puede leer los hashes, ni la dueña.
- En la app:** menú por rol y además cada página verifica el rol en el servidor (entrar por URL directa no sirve). Streamlit no expone una API REST. Bloqueo tras 5 intentos fallidos, mismo mensaje si el usuario no existe, errores sin detalles internos.
- Infraestructura:** la app no tiene la contraseña de administrador; puertos solo en `localhost`; acceso externo solo por HTTPS; secretos en `.env` fuera de git. **Probado:** 32 verificaciones en `db/pruebas/permisos.sql` y 31 pruebas de la app.
- Con más tiempo:** segundo factor para la dueña, bitácora de quién cambió qué, respaldos diarios cifrados, límite de intentos por IP.

6. Cómo usé la IA

Usé **Claude Code** como agente en la terminal. Yo decidí el stack (Postgres y Streamlit en Docker, sin costos), las prioridades y los supuestos; le pedí el plan, el esquema y las políticas de permisos, el script de limpieza, las páginas y las pruebas, y revisé cada cambio antes de hacer commit. Al ver la primera versión pedí un **rediseño** (sin emojis, caja tipo punto de venta, cobranza desde la tabla de clientes, subpestañas): se hizo en una branch aparte, lo revisé en el navegador y lo uní cuando lo aprobé. **Lo que generó mal y corregimos:**

- Escribió a mano los teléfonos de los clientes y le puso a Fernanda Garza el de Jorge Salinas → ahora se toman del Excel y el script reporta teléfonos compartidos.
- Interpretó "Factura: no" como "no requiere factura" → se cambió a "sin factura".
- Las pestañas y el encargo abierto se reiniciaban al recargar → la vista y el encargo se guardan en la sesión. Textos con dos "\$" salían como fórmula → `md()` escapa el signo.
- En el rediseño:** el menú superior de Streamlit 1.65 no se dibujaba → barra de menú propia. Y "Por entregar" fallaba si había encargos pendientes (una variable pisaba a otra) → corregido y cubierto por una prueba.
- Cómo lo verifiqué:** pruebas contra la base real (comparan con la base, no con cifras fijas), recorrido en navegador en escritorio y celular, y cifras cruzadas: la venta calculada por dos caminos, y el saldo vencido en Python = vista SQL.